

Yes — I understand the end goal now. Your handwritten diagram is not really a “Jev app”; it’s a **prompt/plan optimization system where Jev acts as the critic/evaluator and Claude acts as the planner/re-writer.**

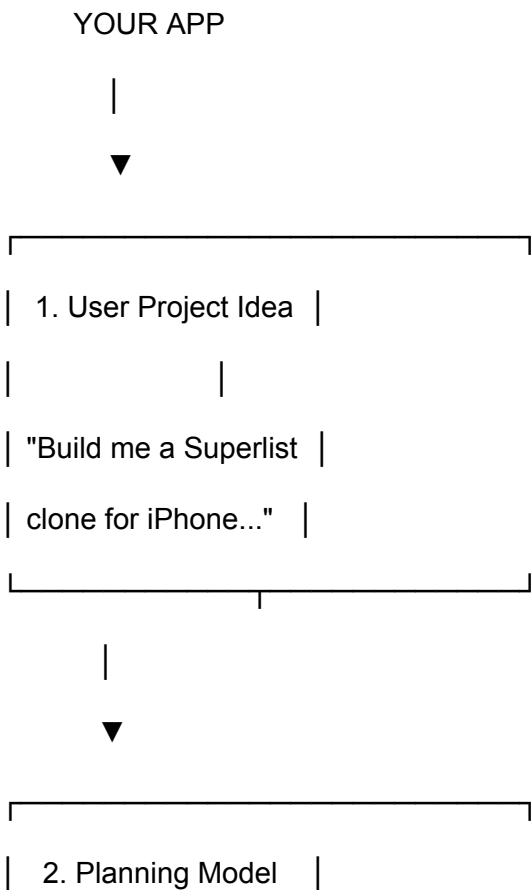
The core loop I see is:

User describes what they want to build → Claude produces an initial build plan → Jev evaluates that plan against a defined scoring matrix → Claude receives the Jev evaluation and rewrites the plan/prompt to fix the weaknesses → user gets a much stronger final prompt/plan.

That division of labor fits Jev particularly well. Jev is designed for bounded semantic judgments—yes/no probabilities, predefined choices, and rubric-based scores—rather than generating plans or explanations itself. Pasted markdown The analysis also explicitly says that more sophisticated behavior can be built by asking Jev atomic questions and letting code or another reasoning model combine the results. Pasted markdown

What I think your product should actually become

I would formalize your handwritten flow like this:



| Claude Opus |

| |

| Produces V1 plan / |

| implementation prompt |

|

|



|

| 3. Jev Critic |

| |

| Evaluates V1 against |

| structured rubrics |

|

|

STRUCTURED SCORES

|



|

| 4. Deterministic |

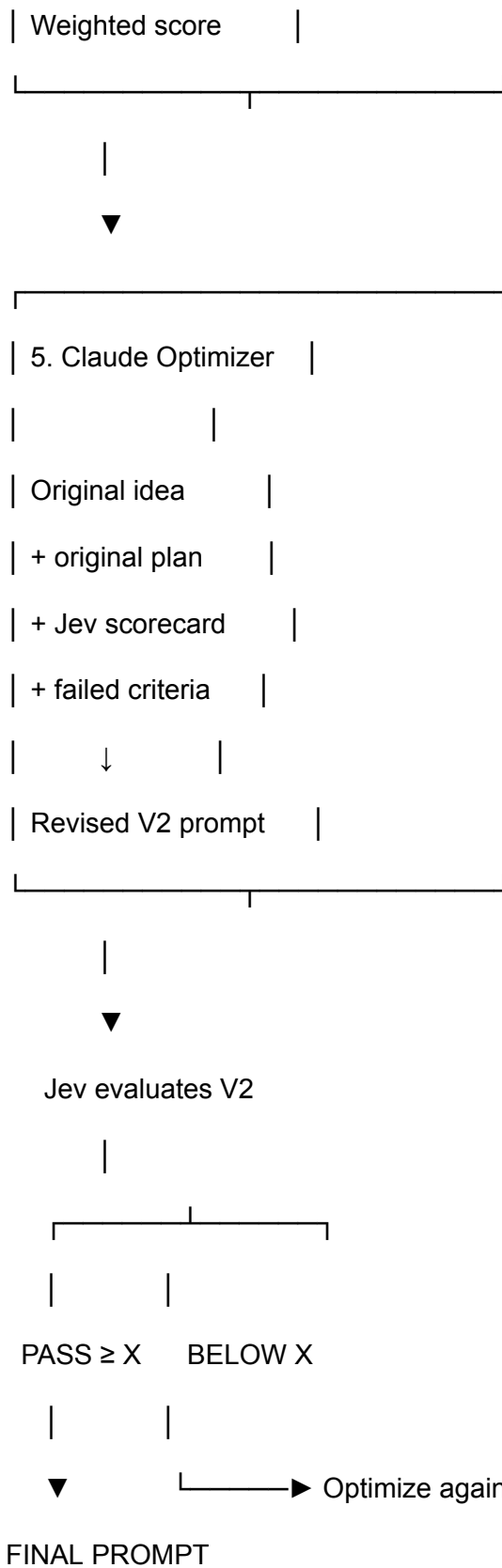
| Evaluation Engine |

| |

| Thresholds |

| Missing requirements |

| Critical failures |



And **this last artifact is the product**: not merely “here is some AI feedback,” but a final, implementation-ready prompt that a user can paste into Claude Code, Cursor, Codex, etc.

The important thing I would change from your handwritten version is this:

Don’t ask Jev for one generic “quality score.”

Your own research warns against interpreting a Jev score as an exact numerical measurement; Jev is better suited to explicit qualitative rubrics, with the actual aggregation and arithmetic handled by your application. Pasted markdown

So your scoring system should look more like this.

Evaluation dimension	Jev question
Requirements coverage	Does this plan cover all explicit requirements in the user's request?
Missing requirements	Which best describes coverage: <code>complete</code> / <code>minor_gaps</code> / <code>major_gaps</code> / <code>insufficient</code>
Architecture	Does the architecture adequately support the requested product?
Implementation specificity	Are instructions specific enough for a coding agent to implement without major guessing?
Data model	Does the proposed data model cover the requested behaviors?
UI coverage	Does the plan describe every major required view and interaction?
Edge cases	How well does the plan address realistic failure and edge cases?

Testing	Does the plan contain sufficient verification criteria?
Security/privacy	Are relevant security/privacy requirements handled?
Dependency correctness	Are the technologies and dependencies internally coherent?
Ambiguity	How much unresolved ambiguity remains?
Build readiness	Could a coding agent begin implementation from this plan without significant additional product decisions?

Then your application turns those outputs into something like:

PROJECT READINESS: 78/100

Requirements	96
Architecture	92
Implementation	73
UI specification	84
Data model	91
Edge cases	52
Testing	44
Security	71
Build readiness	69

CRITICAL FAILURES

- × No migration strategy
- × No offline conflict behavior specified
- × Authentication states underspecified
- × Acceptance tests missing
- × Error/loading/empty states incomplete

The **78/100** is **calculated by your code**, not directly trusted as some magical Jev prediction. That distinction is important because your research notes that a valid bounded output does not guarantee a correct judgment, and confidence can still be high when the supplied categories are inadequate. Pasted markdown Pasted markdown

Then Claude gets something very explicit:

You are the optimization stage.

ORIGINAL USER REQUEST:

...

CURRENT IMPLEMENTATION PLAN:

...

EVALUATION:

Overall: 78/100

FAILED CRITERIA:

- Testing completeness: 44/100
- Edge-case coverage: 52/100
- Build readiness: 69/100

SPECIFIC FAILURES:

1. No explicit migration strategy.
2. Offline conflict resolution unspecified.
3. Authentication failure states missing.
4. No acceptance tests for core flows.
5. Loading/error/empty states incomplete.

TASK:

Rewrite the implementation plan so that every identified deficiency is explicitly resolved.

Do not remove requirements already covered correctly.

Do not arbitrarily expand product scope.

Make requirements testable.

Specify files/components/data models where appropriate.

Return the complete replacement prompt rather than a patch.

And then **Jev evaluates that new version again.**

That feedback loop is the interesting part of your startup.

The bigger product idea

I would take your concept one step further.

Don't make the product:

“Jev scores your Claude prompts.”

Make it:

“Turn an idea into an implementation specification that an AI coding agent can reliably execute.”

Jev becomes infrastructure underneath it.

A user shouldn't even need to know Jev exists.

For example:

I want to build an expense tracker for iPhone.

Your app could progressively transform that into:

idea



requirements specification



architecture



implementation plan



Jev evaluation



gap detection



AI revision



Jev re-evaluation



acceptance criteria



FINAL MASTER BUILD PROMPT

Then the UI could show:

Expense Tracker

Build Readiness



Requirements 98

Architecture 95

Data Model 96

UI Coverage 94

Edge Cases 88

Testing 91

Security 90

✓ 37 requirements verified

✓ 14 edge cases covered

✓ 23 acceptance tests defined

✓ Architecture consistent

2 recommendations remaining

[Optimize Again]

[Open in Claude]

[Open in Cursor]

[Copy Build Prompt]

That's considerably more compelling than a prompt-score website.

And there is another thing from your Jev research that I think should become a fundamental architectural decision: **Jev should be replaceable**. Your analysis made the same observation for a different application—the enduring value should live in the workflow, accumulated evaluations, calibrated thresholds, and product data rather than dependence on the model itself. Pasted markdown

For your startup, the defensible asset eventually becomes something like:

thousands/millions of plans → evaluator results → implementation outcomes → what problems actually caused coding agents to fail → what revisions fixed those failures.

Eventually you can learn:

“When an iOS coding plan looks like *this*, Cursor tends to fail during persistence implementation.”

or:

“Plans missing these 4 characteristics require 2.6× more corrective prompts.”

That is much more interesting than simply having a clever prompt template.

What I would build first

You do **not** need the complete platform yet. The first MVP could literally do one thing extremely well:

Idea → Claude plan → Jev evaluation → Claude revision → Jev re-evaluation → final master prompt.

Don't start with teams, billing, projects, collaboration, ten coding agents, IDE integrations, or dozens of evaluator types.

First prove this:

USER'S ORIGINAL IDEA



BASELINE

61 / 100



OPTIMIZE



VERSION 2

84 / 100



OPTIMIZE



VERSION 3

94 / 100



READY TO IMPLEMENT

And, critically, later test whether **94/100 plans actually produce better software than 61/100 plans**. That is the equivalent of the distinction in your research between semantic judgment and real-world outcome: a model saying something looks strong isn't itself evidence that it performs better. Pasted markdown

So yes: **I now understand the end goal you're aiming at.**

And I think the next thing we should design is **the Jev scoring matrix itself**—the exact questions, scoring rubrics, weights, critical-failure rules, and pass thresholds that sit between Claude V1 and Claude V2. That is essentially the brain of this product.